L I M E

LIME IS MOTION, EXCLUSIVE

TÀI LIỆU ĐẶC TẢ KỸ THUẬT • BẢN 1.0

Đặc tả thanh toán VietQR + SePay cho LIME

Hệ thống thu tiền qua QR chuyển khoản ngân hàng, đối soát tự động bằng webhook biến động số dư. Đây là nguồn sự thật duy nhất để dev triển khai, QA kiểm thử và admin vận hành — viết trước khi code.

149.000đ	15 phút	~95%	7 lần
Gói Sáng tạo / tháng (giá lấy từ bảng plans)	Thời hạn của 1 order và mã QR	Giao dịch khớp tự động còn lại vào hàng duyệt tay	SePay retry webhook giãn cách Fibonacci

Dự án	LIME — nền tảng tạo video từ prompt
Phạm vi	Module billing VietQR: tạo order, sinh QR, nhận webhook SePay, nâng gói, đối soát và xử lý ngoại lệ
Stack hiện tại	Backend Fastify (Node.js), hàng đợi BullMQ, kiểm tra quota trong pipeline tạo video
Ngày ban hành	13/10/2026 — Bản 1.0
Trạng thái	Đã chốt kiến trúc và các edge case. Chưa code — tài liệu này dùng để bắt đầu triển khai.

> **Nguyên tắc cốt lõi, đọc trước khi code:** (1) Số tiền và giá gói chỉ lấy từ server, không bao giờ tin client hay nội dung webhook để quyết định giá. (2) Đối soát chính xác tuyệt đối — sai tiền, sai mã thì không tự kích hoạt. (3) Mọi webhook đều phải idempotent. (4) Phần máy không xử lý được thì đẩy vào hàng chờ admin, không để mất tiền oan hay kích hoạt oan.

Mục lục và cách dùng tài liệu

Mục	Nội dung	Ai cần đọc kỹ
1	Mục tiêu, phạm vi, thuật ngữ	Tất cả
2	Kiến trúc tổng thể và các thành phần	Backend, Frontend
3	Mô hình dữ liệu	Backend
4	Đặc tả API	Backend, Frontend, QA
5	Luồng nghiệp vụ chính	Tất cả
6	Đặc tả webhook SePay	Backend, QA
7	Đối soát, idempotency và cron bù	Backend, Admin
8	Ba case chính: trùng, sửa tiền, trừ tiền chưa nhận	Tất cả
9	14 case phụ theo 4 giai đoạn	Backend, QA, Admin
10–12	Bảo mật, vận hành, kiểm thử, checklist nghiệm thu	Tất cả

1. Mục tiêu, phạm vi, thuật ngữ

Mục tiêu. Cho phép người dùng LIME thanh toán gói trả phí bằng cách quét mã VietQR từ bất kỳ app ngân hàng nào, tiền vào thẳng tài khoản ngân hàng của LIME, hệ thống tự động nâng gói và cộng quota mà không cần cổng thanh toán trung gian, không cần giấy tờ doanh nghiệp như VNPay production hay Stripe.

Trong phạm vi bản 1.0: thanh toán một lần cho một kỳ (30 ngày) của một gói; tạo order và QR; nhận và xử lý webhook SePay; nâng gói, cộng dồn hạn; hàng đối soát tay cho admin; cron hết hạn và cron đối soát bù.

Ngoài phạm vi bản 1.0: tự động trừ tiền định kỳ (auto-renew thật sự — VietQR không hỗ trợ, người dùng phải quét lại mỗi kỳ); hoàn tiền tự động (chuyển khoản ngân hàng không có API refund — hoàn tay); hóa đơn VAT tự động; đa tiền tệ (chỉ VND, số nguyên).

Thuật ngữ	Định nghĩa
VietQR	Chuẩn QR chuyển khoản NAPAS. QR chỉ chứa thông tin gợi ý: số tài khoản, số tiền, nội dung. App ngân hàng được phép sửa số tiền trước khi chuyển.
SePay	Dịch vụ lắng nghe biến động số dư tài khoản ngân hàng liên kết, bắn webhook về server khi có tiền vào/ra. LIME dùng hướng Webhooks, không dùng cổng thanh toán của SePay.
pay_code	Mã thanh toán duy nhất của một order, ví dụ `LIME8X2K9Q` (prefix `LIME` + 6 ký tự). Nằm trong nội dung chuyển khoản để đối soát.
reference_code	Mã giao dịch của ngân hàng (mã FT) do SePay gửi kèm. Là khóa idempotency chính.
unmatched	Giao dịch tiền đã vào nhưng không khớp order nào (sai tiền, sai/thiếu mã, order hết hạn). Chờ admin xử lý tay.

2. Kiến trúc tổng thể

Hệ thống gồm bốn thành phần. Nguyên tắc phân chia: frontend không quyết định tiền, backend không phụ thuộc API ngoài ở bước sinh QR, và webhook là đường tiền vào duy nhất được tự động nâng gói.

Thành phần	Trách nhiệm	Không được làm
Frontend Trang Pricing + modal thanh toán	Hiện QR, số tiền, nội dung chuyển khoản và nút copy; đếm ngược 15 phút; poll trạng thái order mỗi 3 giây; hiện nút "Tôi đã chuyển khoản"; đóng modal ngay khi order paid.	Tự tính giá, tự suy đoán đã thanh toán, tự gia hạn order.
Backend Fastify — module billing	Tạo order và pay_code; sinh QR; nhận webhook; đối soát; nâng gói trong transaction; ghi ledger; chạy cron hết hạn và đối soát bù.	Tin amount do client gửi lên để ghi nhận thanh toán.
SePay	Liên kết tài khoản ngân hàng nhận tiền của LIME; cấu hình IPN URL và API key; đặt pattern mã thanh toán (prefix `LIME`); bắn webhook khi có biến động.	— (dịch vụ ngoài, chỉ cấu hình)
Database	Lưu plans, orders, payments, unmatched và trạng thái gói của user. Các ràng buộc unique trong DB là lớp chống trùng cuối cùng.	Dựa vào logic ứng dụng thôi để chống trùng — phải có unique constraint.

Sinh QR: ưu tiên offline, không phụ thuộc API ngoài

Thứ tự ưu tiên: (1) Sinh chuỗi VietQR chuẩn NAPAS ngay trong Node bằng thư viện local (ví dụ `vietqr` / `uni-vqr`) rồi vẽ QR tại server hoặc client. (2) Chỉ khi cần ảnh QR sẵn mới dùng dịch vụ ảnh như `qr.sepay.vn/img` với tham số tài khoản, ngân hàng, số tiền và nội dung. Cách (1) là mặc định vì nếu dịch vụ ảnh sập, luồng thanh toán vẫn sống.

Cấu hình bắt buộc (biến môi trường)

Biến	Ý nghĩa
`SEPAY_API_KEY`	Khóa xác thực webhook và gọi SePay Transactions API. Chỉ ở server, không lộ ra frontend, không ghi log.
`BANK_ACCOUNT` / `BANK_CODE`	Số tài khoản và ngân hàng nhận tiền, dùng để sinh QR. Đổi tài khoản phải đi kèm quy trình ở Mục 9, case 14.
`PAY_CODE_PREFIX`	Mặc định `LIME`. Phải khớp pattern đã cấu hình trên SePay.
`ORDER_TTL_MINUTES`	Mặc định 15. Hết hạn thì order chuyển `expired`.
`CLAIMED_EXTEND_MINUTES`	Mặc định 30. Gia hạn chờ đối khớp khi user bấm "Tôi đã chuyển khoản".

> **Phí:** SePay tính phí cố định theo giao dịch (khoảng 200–500đ/giao dịch tùy gói dịch vụ) thay vì phần trăm doanh thu. Khi đổi giá gói hoặc đổi nhà cung cấp, cập nhật con số này trong vận hành, không hardcode vào logic đối soát.

3. Mô hình dữ liệu

Năm bảng/cột bổ sung. Mọi thay đổi trạng thái quan trọng đều ghi thời điểm và để lại vết trong ledger — không cập nhật xóa vết.

Bảng	Cột chính và ràng buộc
`plans`	`id, name, price_vnd (int, VND), quota_video, days (mặc định 30), is_active`. Đây là nguồn giá duy nhất. Ví dụ: Gói Sáng tạo — 149000 — 30 ngày.
`orders`	`id, user_id, plan_id, amount_vnd, pay_code UNIQUE, status, expires_at, claimed_at, paid_at, created_at`. `amount_vnd` sao chép từ `plans.price_vnd` tại thời điểm tạo, để đổi giá sau này không làm sai order cũ.
`payments` (ledger)	`id, order_id, sepay_ref UNIQUE, amount_vnd, raw (json gốc webhook), received_at, result`. `result` gồm: `matched, duplicate, underpaid, overpaid, unmatched`. `sepay_ref UNIQUE` là khóa idempotency bắt buộc.
`unmatched_payments`	`id, sepay_ref UNIQUE, amount_vnd, content, raw, status (pending/resolved/refunded), admin_note, resolved_by, resolved_at`. Mọi giao dịch không khớp đều vào đây, không được bỏ qua.
`users` (bổ sung)	`plan_id, plan_expires_at`. Nâng gói: `plan_expires_at = max(now, plan_expires_at hiện tại) + plan.days` — cộng dồn, không reset.

Trạng thái order

Trạng thái	Ý nghĩa	Chuyển sang
`pending`	Vừa tạo, chờ chuyển khoản, QR còn hạn	`paid` khi khớp đúng; `claimed` khi user báo đã chuyển; `expired` hết 15 phút; `cancelled` khi user tạo order mới / admin hủy
`claimed`	User đã bấm "Tôi đã chuyển khoản", đang chờ ngân hàng xác nhận. Được gia hạn thêm 30 phút chờ đối khớp	`paid` khi webhook khớp; quay về hết hạn nếu quá thời gian gia hạn mà không khớp
`paid`	Đã khớp tiền đúng và đã nâng gói. Trạng thái cuối, không sửa ngược	Không chuyển nữa. Giao dịch trùng sau đó ghi ledger `duplicate`
`expired`	Hết hạn mà chưa khớp	Tiền vào sau hết hạn không mở lại order — vào `unmatched`
`cancelled`	Bị hủy do tạo order mới, đổi tài khoản nhận, hoặc admin hủy	—

> **Quy tắc 1 order pending / user:** Khi tạo order mới, hủy các order `pending` cũ của cùng user (hoặc tái sử dụng nếu cùng gói và còn hạn — chọn một cách và làm nhất quán). Khi một order của user chuyển `paid`, tự động hủy các order pending còn lại của user đó.

4. Đặc tả API

Tất cả endpoint billing yêu cầu đăng nhập (trừ webhook). Lỗi trả về kèm mã lỗi ổn định để frontend hiện đúng thông điệp và QA viết test.

4.1 Tạo order — `POST /api/billing/vietqr/order`

Trường	Mô tả
Request	`{ "plan_id": "..." }` — chỉ nhận plan_id. Không nhận amount từ client.
Xử lý	Đọc giá từ `plans`; hủy pending cũ của user; tạo pay_code random duy nhất (kiểm tra UNIQUE, sinh lại nếu trùng); `expires_at = now + 15 phút`; sinh QR offline.
Response 200	`{ order_id, pay_code, amount_vnd, qr_content / qr_image, expires_at, bank_account, bank_code }`
Lỗi	`401` chưa đăng nhập; `404 PLAN_NOT_FOUND`; `400 PLAN_INACTIVE`.

4.2 Trạng thái order — `GET /api/billing/vietqr/order/:id`

Frontend poll mỗi 3 giây khi modal đang mở. Chỉ chủ order mới xem được (`403` nếu không phải chủ). Response: `{ status, amount_vnd, pay_code, expires_at, plan_expires_at }`. Dừng poll ngay khi `status = paid` hoặc modal đóng.

4.3 Báo đã chuyển — `POST /api/billing/vietqr/order/:id/claimed`

Chỉ có tác dụng khi order đang `pending`: chuyển sang `claimed`, ghi `claimed_at`, gia hạn chờ đối khớp thêm 30 phút, trả về trạng thái "đang chờ ngân hàng xác nhận". Bấm nhiều lần không gia hạn vô hạn — tối đa 1 lần gia hạn / order. Endpoint này không nâng gói và không ghi nhận thanh toán.

4.4 Webhook SePay — `POST /api/billing/vietqr/webhook`

Trường	Mô tả
Xác thực	Header `Authorization: Apikey <SEPAY_API_KEY>`. Sai key: trả `401`, ghi log cảnh báo, không xử lý. Tăng cường: IP allowlist của SePay (Mục 10).
Body vào	JSON của SePay, các trường dùng để đối soát: `transfer_type` (chỉ xử lý `"in"`), số tiền vào, `content` (nội dung chuyển khoản chứa pay_code), `code` (mã SePay bóc tách, nếu có), `reference_code` (mã FT).
Response thành công	HTTP 200 và body đúng `{"success": true}` trong dưới 30 giây. Cả hai điều kiện đều bắt buộc (chi tiết Mục 6).
Idempotency	Nếu `reference_code` đã tồn tại trong `payments`: không xử lý lại, vẫn trả `{"success": true}`.

> **Cấm:** Không dùng số tiền trong webhook để "quyết định giá gói". Số tiền chỉ để so khớp với `orders.amount_vnd` đã lưu từ trước. Lệch là không khớp, không có ngoại lệ tự động.

5. Luồng nghiệp vụ chính (happy path)

1. Bấm Nâng cấp
User chọn gói ở trang Pricing. Frontend gọi tạo order.

2. Tạo order
Server tạo pending, pay_code riêng, hết hạn +15 phút.

3. Hiện QR
Modal hiện QR + tiền + nội dung + đếm ngược. Poll 3 giây/lần.

4. Quét và chuyển
User quét bằng app ngân hàng, giữ nguyên tiền và nội dung.

5. Webhook về
SePay thấy tiền vào, POST về server. Server đối soát.

6. Nâng gói
Khớp đúng: order paid, ghi ledger, nâng plan, cộng quota.

Điều kiện khớp (tất cả phải đúng cùng lúc): (a) tìm thấy pay_code trong nội dung chuyển khoản sau khi chuẩn hóa; (b) order đang `pending` hoặc `claimed` và chưa quá thời gian chờ khớp; (c) số tiền webhook bằng `orders.amount_vnd` chính xác, không dung sai; (d) `reference_code` chưa từng xử lý. Khớp thì thực hiện trong một transaction database: khóa dòng order, cập nhật `paid`, ghi `payments`, nâng `users.plan_id / plan_expires_at`, cộng quota. Lỗi giữa chừng: transaction rollback và có job quét bù (Mục 7).

Chuẩn hóa nội dung để tìm mã: viết hoa toàn bộ, bỏ khoảng trắng và ký tự thừa, tìm pattern prefix + 6 ký tự. Nếu trường `code` của SePay không khớp, fallback tự parse trường `content` thô trước khi kết luận unmatched (SePay có thể bóc sai khi user gõ thừa dấu/ký tự).

6. Đặc tả webhook SePay — retry và phản hồi

Tham số	Giá trị đã xác minh
Số lần retry tối đa	7 lần
Giãn cách retry	Fibonacci: 1, 1, 2, 3, 5, 8, 13 phút — tổng cửa sổ khoảng 33 phút
Định nghĩa "thành công"	HTTP 200 **và** body đúng `{"success": true}`, phản hồi trong 30 giây
Trả 200 nhưng body sai	Bị tính là thất bại nhưng **không được retry nữa** — đây là bẫy nguy hiểm nhất, phải test kỹ
Webhook quá 5 giờ	SePay ngừng retry hẳn. Chỉ còn cách bấm "Gọi lại" tay trên dashboard SePay, hoặc cron đối soát bù bắt được (Mục 7)
Server down lúc webhook tới	An toàn nhờ retry + idempotency. Xử lý trùng lặp không được nâng gói hai lần

Hệ quả thiết kế: Handler webhook phải nhẹ và nhanh — xác thực, ghi nhận, đối soát đồng bộ trong transaction ngắn, trả `{"success": true}` ngay. Việc nặng (gửi email, thông báo, tính toán phụ) đẩy sang hàng đợi BullMQ xử lý sau, không để webhook chờ quá 30 giây.

7. Đối soát, idempotency và cron bù

7.1 Ba lớp chống trùng

1. **Unique constraint:** `payments.sepay_ref UNIQUE` và `unmatched_payments.sepay_ref UNIQUE`. Giao dịch đã ghi thì không ghi lại, dù webhook retry bao nhiêu lần.
2. **Khóa dòng:** Khi đối soát, khóa dòng order (SELECT … FOR UPDATE hoặc tương đương) để hai webhook chạy song song không cùng nâng gói một order.
3. **Kiểm tra trạng thái trong transaction:** Chỉ nâng gói nếu order đang `pending/claimed` tại thời điểm khóa. Order đã `paid` thì giao dịch mới ghi `duplicate`.

7.2 Cron đối soát bù — không phụ thuộc hoàn toàn vào webhook

Job	Tần suất	Việc làm
Expire orders	Mỗi 1 phút	Order `pending` quá `expires_at` (và `claimed` quá thời gian gia hạn) chuyển `expired`.
Đối soát bù SePay	Mỗi 5–15 phút	Gọi SePay Transactions API liệt kê giao dịch tiền vào gần nhất, lọc giao dịch webhook thất bại (`webhook_success = 0`), chạy lại đúng pipeline đối soát như webhook. Giao dịch đã có `sepay_ref` thì bỏ qua.
Quét paid chưa nâng gói	Mỗi 5–15 phút	Tìm order `paid` nhưng `users.plan` chưa cập nhật tương ứng (lỗi giữa transaction / lỗi DB) — tự vá và alert admin.
Cảnh báo tồn đọng	Mỗi 15–30 phút	Alert nếu có `unmatched` chưa xử lý, order `claimed` quá lâu không khớp, hoặc duplicate chờ hoàn tiền.

7.3 Nguyên tắc đối soát số tiền

Tình huống	Kết quả tự động
Đúng mã + đúng tiền	Khớp. Order `paid`, nâng gói, cộng quota.
Đúng mã + thiếu tiền	**Không kích hoạt.** Giữ tiền đã nhận, ghi ledger, báo user chuyển bù đúng mã nội dung cho đủ. Xử lý tiếp ở Mục 8.2.
Đúng mã + thừa tiền	Kích hoạt gói bình thường (đã trả đủ), đồng thời mở ticket hoàn phần thừa cho admin.
Sai / thiếu mã, hoặc sai tiền không tìm thấy order	Vào `unmatched_payments` + alert admin. Không tự đoán gán cho user nào.

Lưu ý nhất quán: "Thừa tiền" ở đây nghĩa là số tiền lớn hơn giá gói nhưng kèm đúng mã — kích hoạt + hoàn phần thừa. "Thiếu tiền" không bao giờ kích hoạt, kể cả thiếu 1 đồng.

7.4 Kiểm thử nhanh mục này

- Gửi cùng một webhook hai lần liên tiếp: chỉ một lần nâng gói, lần hai trả success mà không ghi trùng.
- Tắt server, để SePay retry, bật lại: giao dịch vẫn được ghi nhận đúng một lần.
- Mô phỏng webhook thất bại rồi chạy cron bù: giao dịch được bắt lại và đối soát đúng như webhook realtime.

8. Ba case chính

8.1 Hai máy cùng quét một mã — trùng thanh toán

Tình huống. Mã QR gắn với đúng một order của một user (mở modal trên điện thoại và laptop, hoặc chia sẻ ảnh QR). Cả hai đều chuyển đủ tiền, đúng nội dung.

Xử lý. Giao dịch tới trước khớp và nâng gói. Giao dịch thứ hai thấy order đã `paid`: không nâng gói lại, ghi `payments.result = duplicate` kèm `reference_code` riêng, tạo ticket/alert cho admin hoàn tiền thủ công. Phân biệt các giao dịch bằng `reference_code` (mã FT) — không bao giờ gộp hai giao dịch làm một.

Giảm thiểu. Modal ghi rõ "Mã này chỉ dùng cho tài khoản của bạn, không chia sẻ"; order paid thì đóng modal và dừng poll ngay; hai user khác nhau không bao giờ chung mã vì pay_code sinh random và UNIQUE.

> **Giới hạn phải chấp nhận:** Chuyển khoản ngân hàng không có hoàn tiền tự động như Stripe. Duplicate chỉ xử lý thủ công, nên mục tiêu là giảm xác suất xảy ra bằng UX, không phải tự động hóa việc hoàn.

8.2 User quét rồi sửa số tiền

Tình huống. QR chỉ gợi ý số tiền; app ngân hàng cho sửa thoải mái trước khi xác nhận. User sửa xuống (hoặc lên) rồi chuyển.

Xử lý. Webhook về với số tiền khác `orders.amount_vnd` thì không khớp. Thiếu tiền: order vẫn `pending` tới khi hết hạn, tiền giữ lại, thông báo user chuyển bù đúng mã nội dung; nếu không bù thì vào quy trình unmatched/hoàn tay. Thừa tiền kèm đúng mã: kích hoạt gói + ticket hoàn phần thừa. Tuyệt đối không nới logic thành "gần đúng cũng chấp nhận" — chỉ cần chấp nhận thiếu là kẻ gian chuyển 1.000đ cũng chiếm được gói.

Giảm thiểu. Trong modal nhấn mạnh hai dòng: "Giữ nguyên số tiền" và "Giữ nguyên nội dung chuyển khoản". Hiện số tiền và nội dung to, có nút copy riêng.

8.3 Khách bị trừ tiền nhưng LIME chưa nhận được

Nguyên nhân thường gặp: webhook SePay chậm vài giây đến vài phút; server downtime đúng lúc webhook tới; user chọn "chuyển thường" thay vì chuyển nhanh 24/7 nên tiền tới sau vài giờ; user chuyển nhầm tài khoản (SePay không bao giờ thấy giao dịch này).

Xử lý. Nút "Tôi đã chuyển khoản" chuyển order sang `claimed` + gia hạn 30 phút + hiện trạng thái chờ xác nhận, tránh để user hoang mang tạo giao dịch thứ hai. Webhook idempotent nên tiền về muộn vẫn khớp đúng. Cron cảnh báo order `claimed` quá lâu và unmatched để admin đối chiếu dashboard SePay / sao kê. Trường hợp nhầm tài khoản: chỉ xử lý bằng biên lai user gửi + đối chiếu thủ công, hệ thống không tự làm được.

Thông điệp cho user trong modal: dùng chuyển nhanh 24/7; sau khi chuyển, chờ tối đa vài phút; nếu quá 30 phút chưa thấy kích hoạt thì gửi biên lai cho hỗ trợ thay vì chuyển lại lần hai.

9. Mười bốn case phụ — theo giai đoạn

Các case dưới đây suy ra từ kiến trúc đã chốt ở Mục 1–8 để làm checklist triển khai và test. Mỗi case nêu rõ: tình huống — cách xử lý.

Giai đoạn A: Lúc tạo QR / user chưa chuyển

#	Case	Xử lý
1	Spam nút Nâng cấp, tạo nhiều order pending	Giới hạn 1 pending/user (Mục 3). Tạo mới thì hủy cũ; một order paid thì hủy các pending còn lại.
2	QR hết hạn 15 phút nhưng user vẫn chuyển	Tiền vào, order đã `expired` — không mở lại tự động. Vào `unmatched`, admin quyết định gia hạn thủ công hoặc hoàn.
3	Dịch vụ sinh ảnh QR (SePay/vietqr) sập	Fallback sinh QR offline bằng lib local là mặc định (Mục 2), nên case này không chặn được luồng thanh toán.

Giai đoạn B: Lúc chuyển khoản

#	Case	Xử lý
4	Gộp 2 kỳ vào 1 giao dịch (ví dụ 298.000đ)	Số tiền không khớp bất kỳ order nào — vào `unmatched`. Admin tách tay: kích hoạt 1 kỳ và xử lý phần còn lại, hoặc hoàn. Bản 1.0 không tự tách.
5	Giao dịch NAPAS treo / timeout, tiền tự hoàn (reversal)	Webhook không bao giờ tới vì tiền không vào tài khoản. Hướng dẫn user kiểm tra sao kê phía họ nếu app báo "đang chờ" quá lâu; không ghi nhận gì ở phía LIME.
6	Chuyển nhầm tài khoản (tự nhập tay thay vì quét)	SePay không thấy giao dịch. Chỉ xử lý bằng biên lai + đối chiếu thủ công. Giảm thiểu bằng QR to, nút copy số tài khoản/nội dung.
7	Ngân hàng bảo trì nửa đêm, chuyển thất bại phía user	Không phải lỗi hệ thống LIME. Order vẫn pending trong hạn; UX báo thử lại sau, đừng tạo order mới liên tục.

Giai đoạn C: Lúc webhook về

#	Case	Xử lý
8	Webhook giả mạo (attacker tự POST)	Chặn bằng verify `Authorization: Apikey` + IP allowlist. Giá lấy từ `plans` phía server; không tin số tiền webhook để quyết định (Mục 10).
9	Webhook retry tới sau khi đã xử lý (response lần đầu bị mất)	Idempotency bằng `reference_code UNIQUE`: trả `{"success": true}` và không xử lý lại.
10	SePay bóc sai mã từ nội dung (user gõ thừa dấu/ký tự)	Fallback tự parse `content` thô tìm pattern pay_code trước khi kết luận unmatched (Mục 5).

Giai đoạn D: Sau khi paid / vận hành

#	Case	Xử lý
11	Order đã paid nhưng nâng gói fail giữa chừng (lỗi DB)	Job quét "paid nhưng user chưa có plan" tự vá + alert (Mục 7.2). Thiết kế transaction đúng sẽ hiếm khi xảy ra, nhưng vẫn phải có lưới an toàn.
12	Gia hạn khi gói còn hạn	Cộng dồn: `plan_expires_at = max(now, hạn cũ) + 30 ngày`. Không reset về 0, không trừ ngày còn lại.
13	User đổi ý / đòi hoàn tiền	Quy trình tay: hoàn tiền qua ngân hàng, trừ quota/plan tương ứng, ghi log đầy đủ (ai hoàn, bao nhiêu, lý do, mã giao dịch). Tranh chấp thì yêu cầu biên lai đối chiếu sao kê.
14	Đổi số tài khoản nhận tiền của LIME	QR pending đang trỏ tài khoản cũ. Chọn một: giữ cả hai tài khoản cùng liên kết SePay một thời gian chuyển tiếp, hoặc hủy toàn bộ pending khi đổi và thông báo user tạo lại. Không đổi âm thầm.

10. Bảo mật

- **Xác thực webhook hai lớp:** verify `Authorization: Apikey` là bắt buộc; IP allowlist theo danh sách IP SePay công bố là lớp tăng cường. Sai xác thực thì trả 401, ghi log, không tiết lộ lý do chi tiết.
- **Không tin dữ liệu ngoài:** Giá từ `plans`; số tiền webhook chỉ để so khớp; nội dung chuyển khoản chỉ để tìm pay_code. Không trường nào trong webhook được phép tạo user, đổi giá hay bỏ qua đối soát.
- **Quyền xem order:** API trạng thái chỉ trả cho chủ order. pay_code không phải bí mật nhưng không được liệt kê công khai.
- **Bí mật:** `SEPAY_API_KEY` chỉ ở biến môi trường server, không đưa vào frontend, không ghi vào log. Log webhook lưu body đã che thông tin nhạy cảm không cần thiết.
- **HTTPS bắt buộc:** IPN/webhook URL phải là HTTPS công khai. Môi trường dev local expose qua ngrok và dùng cùng cơ chế verify như production.
- **Rate limit:** Giới hạn tạo order / user / phút để chống spam pay_code; giới hạn request webhook sai key để chống dò key.

Rủi ro còn lại phải chấp nhận và ghi rõ cho vận hành

VietQR không có cơ chế giữ tiền hay tranh chấp như cổng thanh toán: mọi tranh chấp, hoàn tiền và nhầm tài khoản đều xử lý thủ công dựa trên sao kê. Tài liệu này giảm rủi ro bằng đối soát chặt và hàng unmatched, chứ không loại bỏ được bản chất đó.

10.5 Gian lận và kiểm soát — để dành cho nâng cấp

> **Kết luận cho bản 1.0:** Chỉ dùng VietQR thì **không xây hệ thống fraud detection / risk scoring riêng**. Lớp kiểm soát mỏng ở Mục 10.5.2 — vốn đã nằm rải rác trong đặc tả này — là đủ. Mục này ghi lại lý do và điều kiện kích hoạt, để khi nâng cấp có căn cứ mà làm, không phải việc phải code ngay.

10.5.1 Vì sao VietQR ít bề mặt gian lận: push khác pull

Khía cạnh	Push payment — VietQR (bản 1.0)	Pull payment — thẻ / cổng thanh toán (tương lai)
Ai khởi tạo	Khách tự mở app ngân hàng của mình — đã xác thực danh tính (KYC) bởi ngân hàng — và chủ động đẩy tiền đi.	Người bán / cổng thanh toán cầm thông tin thẻ hoặc token để trừ tiền thay khách.
Rủi ro đặc trưng	Không có chuyện "thẻ lậu" hay chargeback: chuyển khoản ngân hàng gần như không thể đảo ngược, tiền vào là tiền thật. Rủi ro còn lại chỉ là webhook giả, sai tiền / sai mã, spam order.	Thẻ đánh cắp, giao dịch bị tranh chấp, chargeback và hoàn tiền bắt buộc — đây mới là đất của gian lận thanh toán.
Hệ quả thiết kế	Chặn bằng validate và đối soát (Mục 4.4, 5, 7.3, 10) là đủ, không cần chấm điểm rủi ro.	Mới cần fraud detection thật: risk scoring, 3-D Secure, velocity check, hàng review theo điểm rủi ro.

10.5.2 Lớp kiểm soát mỏng đã có — không phải fraud detection

- **Verify webhook** `Authorization: Apikey` + IP allowlist (Mục 4.4, 10): chặn webhook giả mạo — vector tấn công thực tế nhất của luồng này.
- **Đối soát số tiền chính xác tuyệt đối**, giá lấy từ bảng `plans` phía server (Mục 5, 7.3): chuyển thiếu hay sai thì không kích hoạt được, kể cả thiếu 1 đồng.
- **Rate-limit tạo order** + `pay_code` random và UNIQUE (Mục 3, 10): chống spam order và dò mã.
- **Audit log mọi thao tác tay của admin** — gán unmatched, hoàn tiền, kích hoạt tay (Mục 9 case 13, 11.1): truy vết được ai làm gì, chống lạm quyền từ bên trong.
- **Alert khi unmatched / refund tăng đột biến** (Mục 7.2, 11.1): phát hiện sớm bất thường để người xem xét, thay vì để hệ thống tự quyết.

Đây là validate + giám sát vận hành: không chấm điểm giao dịch, không tự từ chối dựa trên mô hình rủi ro, không suy đoán ý định của người chuyển.

10.5.3 Những thứ không làm ở bản 1.0

- Không mua hay tích hợp dịch vụ risk scoring (kiểu Stripe Radar, Sift) cho luồng VietQR: không có tín hiệu thẻ / thiết bị để chấm, chi phí không đổi lại giá trị.
- Không xây mô hình ML tự học, không fingerprint thiết bị, không tự động blacklist IP / tài khoản chỉ dựa trên vài giao dịch lỗi.
- Không nới lỏng đối soát để "giảm false positive": sai tiền, sai mã vẫn vào hàng unmatched duyệt tay như Mục 7.3 và 8.2.

10.5 Gian lận và kiểm soát (tiếp) — lộ trình và cổng phạm vi

10.5.4 Lộ trình khi nâng cấp

Khi nào	Việc cần bổ sung
Thêm thanh toán thẻ (Stripe / VNPay gateway)	Bật fraud detection của chính gateway (Stripe Radar hoặc quy tắc của VNPay), 3-D Secure cho giao dịch thẻ, quy trình xử lý chargeback / dispute kèm bằng chứng giao dịch, và tách ledger thẻ khỏi ledger chuyển khoản. Chỉ lúc này risk scoring mới có dữ liệu (thẻ, BIN, IP, thiết bị) để hoạt động — với VietQR, các tín hiệu đó không tồn tại.
Quy mô lớn / bị tấn công thật	Velocity rules theo user / IP / thiết bị trong cửa sổ thời gian, hàng review theo ngưỡng rủi ro thay vì duyệt đều tay, dashboard theo dõi tỉ lệ dispute và unmatched theo thời gian. Chỉ cân nhắc dịch vụ scoring bên thứ ba khi tỉ lệ gian lận đo được đã chạm cổng ở Mục 10.5.5 — mua trước khi có dữ liệu là trả tiền cho một lớp bảo vệ không có gì để chấm.

10.5.5 Cổng phạm vi — điều kiện để xem xét, không phải cam kết tự động

Chỉ mở bài toán fraud detection khi có ít nhất một điều kiện sau, kèm số liệu hoặc bằng chứng cụ thể:

- LIME bắt đầu nhận thanh toán thẻ / pull payment — rủi ro đổi bản chất, phải đánh giá ngay từ khi thiết kế kênh mới, không chờ xảy ra sự cố.
- Đã ghi nhận vụ gian lận hoặc lạm dụng thực tế (webhook giả lọt qua, lạm dụng hoàn tiền, tấn công tạo order có tổ chức), hoặc ngân hàng / đối tác thanh toán yêu cầu kiểm soát bổ sung.
- Khối lượng giao dịch và số ca ngoại lệ vượt khả năng duyệt tay của admin, hoặc tỉ lệ unmatched / refund bất thường kéo dài dù đã xử lý theo Mục 11.1.

Để các điều kiện trên đo được thay vì cảm tính, ngay từ bản 1.0 hệ thống phải ghi lại (đây là phần mở rộng của audit log và alert ở Mục 10.5.2, không phải hệ thống mới):

- Số webhook bị từ chối vì sai API key / sai IP mỗi ngày — dấu hiệu sớm của dò tấn công.
- Tỉ lệ unmatched, duplicate và refund trên tổng số giao dịch, kèm lý do xử lý của admin.
- Số order bị hủy vì hết hạn / spam theo user, để phân biệt hành vi bất thường với user dùng sai cách.

> **Lưu ý:** Ngưỡng định lượng cụ thể (tỉ lệ dispute, số giao dịch mỗi ngày...) chỉ chốt được sau khi có dữ liệu vận hành thật và yêu cầu của gateway / ngân hàng tại thời điểm nâng cấp — tài liệu này không đặt số cứng từ bây giờ. Khi chạm cổng, bước đầu tiên là đánh giá lại rủi ro và cập nhật mục này, không tự động triển khai scoring.

> **Tóm lại:** Bản 1.0 giữ kiểm soát mỏng và ghi dữ liệu. Fraud detection chỉ trở thành việc cần làm khi kênh thanh toán đổi sang pull payment, hoặc khi số liệu vận hành cho thấy lớp mỏng không còn đủ.

11. Vận hành, giám sát và kiểm thử

11.1 Vận hành hằng ngày (admin)

- **Hàng unmatched:** đối chiếu nội dung/số tiền/biên lai, rồi chọn: gán vào order và kích hoạt tay, yêu cầu chuyển bù, hoặc hoàn tiền. Mọi quyết định ghi `admin_note`.
- **Hàng duplicate / thừa tiền:** hoàn phần trùng/thừa qua ngân hàng, đánh dấu ticket đã hoàn kèm mã giao dịch hoàn.
- **Cảnh báo tự động:** unmatched mới, `claimed` quá 30 phút không khớp, duplicate mới, cron bắt được webhook thất bại. Kiểm tra kênh alert mỗi ngày; đối chiếu tổng `payments` với sao kê mỗi tuần.

11.2 Cách test — giới hạn phải biết

> **SePay không có sandbox giả lập webhook biến động số dư** (không test được như Stripe test mode). Hai cách test được chấp nhận:

1. **Nút "Gửi test webhook" trên dashboard SePay:** test xác thực, định dạng response `{"success": true}`, idempotency khi gửi lại cùng payload, và các nhánh sai tiền/sai mã bằng payload sửa tay.
2. **Chuyển khoản thật số tiền nhỏ:** tạo gói test giá nhỏ (2.000–5.000đ) và chuyển thật để kiểm tra đầu-cuối: QR, webhook, nâng gói, ledger, poll.

11.3 Bộ test case nghiệm thu tối thiểu

ID	Kịch bản	Kết quả mong đợi
T01	Happy path: đúng mã, đúng tiền	Paid, nâng gói, cộng 30 ngày, 1 dòng ledger matched
T02	Gửi lại cùng webhook (cùng reference_code)	Không nâng gói lần 2, vẫn trả success
T03	Hai giao dịch đúng mã liên tiếp	1 matched + 1 duplicate + ticket hoàn
T04	Thiếu tiền 1.000đ	Không kích hoạt, có thông báo chuyển bù
T05	Thừa tiền kèm đúng mã	Kích hoạt + ticket hoàn phần thừa
T06	Sai / thiếu mã nội dung	Vào unmatched, alert admin
T07	Webhook sai API key	401, không ghi nhận gì
T08	Handler trả 200 nhưng body sai	Test phải bắt được lỗi này (SePay sẽ không retry)
T09	Claimed rồi webhook mới tới	Vẫn khớp trong thời gian gia hạn
T10	Order hết hạn rồi tiền mới vào	Unmatched, không tự mở lại order
T11	Gia hạn khi gói còn 10 ngày	Hạn mới = hạn cũ + 30 ngày
T12	Chạy cron đối soát bù	Giao dịch webhook_success=0 được bắt, xử lý đúng

12. Checklist triển khai và tiêu chí nghiệm thu

12.1 Checklist triển khai

- ☐ Tạo bảng `plans / orders / payments / unmatched_payments`, bổ sung `users.plan_id, plan_expires_at`; đủ unique constraint cho `pay_code` và `sepay_ref`.
- ☐ Liên kết tài khoản ngân hàng nhận tiền trên SePay; cấu hình IPN URL (HTTPS), API key và pattern prefix `LIME`.
- ☐ Implement 3 endpoint billing + webhook handler trả đúng `{"success": true}` dưới 30 giây.
- ☐ Sinh QR offline bằng lib local; frontend modal có đếm ngược, nút copy, nút claimed và poll 3 giây.
- ☐ Implement đối soát trong transaction + khóa dòng + chuẩn hóa/fallback parse nội dung.
- ☐ Implement 4 cron: expire, đối soát bù, quét paid chưa nâng gói, cảnh báo tồn đọng.
- ☐ Màn hình admin tối thiểu: danh sách unmatched, duplicate/thừa tiền, claimed quá hạn, và thao tác xử lý kèm ghi chú.
- ☐ Copy trong modal: giữ nguyên tiền + nội dung, dùng chuyển nhanh 24/7, không chuyển lại lần hai khi đang chờ xác nhận.

12.2 Tiêu chí nghiệm thu (Definition of Done)

- 12 test case T01–T12 ở Mục 11.3 đều pass, trong đó T02, T07, T08 là test bảo mật/idempotency bắt buộc.
- Một giao dịch thật số tiền nhỏ chạy đầu-cuối thành công trên tài khoản ngân hàng thật.
- Không có đường nào nâng gói được khi: sai tiền, sai mã, sai API key, hoặc gửi trùng webhook.
- Mọi dòng tiền vào không khớp đều xuất hiện trong hàng unmatched kèm alert — không có giao dịch nào "biến mất" khỏi hệ thống.
- Admin xử lý được unmatched/duplicate mà không cần sửa trực tiếp database.

12.3 Quyết định đã chốt và câu hỏi mở

Đã chốt	Còn mở — cần chủ dự án quyết trước khi code
Hướng VietQR + SePay Webhooks (không dùng cổng SePay, không Stripe/VNPay cho bản này); giá gói đọc từ `plans`; order sống 15 phút, claimed +30 phút; đối soát chính xác tuyệt đối; cộng dồn hạn khi gia hạn.	Ngân hàng và số tài khoản nhận tiền cụ thể; gói dịch vụ SePay và mức phí chính xác; kênh nhận alert admin (email / Telegram / trong dashboard); ai là người duyệt unmatched và hoàn tiền; có gói nào ngoài gói 149.000đ/tháng ở bản 1.0 hay không.

> **Bước tiếp theo sau tài liệu này:** chốt các câu hỏi mở ở trên, rồi mới code module theo đúng thứ tự checklist 12.1. Nếu trong lúc code phát sinh case mới, bổ sung vào Mục 8/9 và tăng phiên bản tài liệu thay vì xử lý âm thầm trong code.

— Hết tài liệu — Bản 1.0, 13/10/2026. LIME is motion, exclusive.